DevOps

People Involved in a DevOps Culture

DevOps is a culture of collaboration and shared responsibility.

Client / Business Team

- Provides requirements
- Defines business goals

Product Owner / Business Analyst

- Understands business needs
- Prioritizes features
- Acts as a bridge between business and teams

Development Team

- Writes code
- Builds features
- Fixes bugs
- Continuously improves the application

QA / Testing Team

- Tests the application
- Finds defects
- Ensures quality and reliability

DevOps Engineers

- Automates build, test and deployment
- Manages CI/CD pipelines
- Manages infrastructure as code
- Enables collaboration between Dev and Ops
- Monitors and improves workflows

Operations / SRE Team

- Manages production environments
- Ensures availability, performance and security
- Monitors systems and handles incidents
- Ensures backups and disaster recovery

End Users

- Uses the application
- Provides feedback
- Helps improve the product

Continuous Feedback

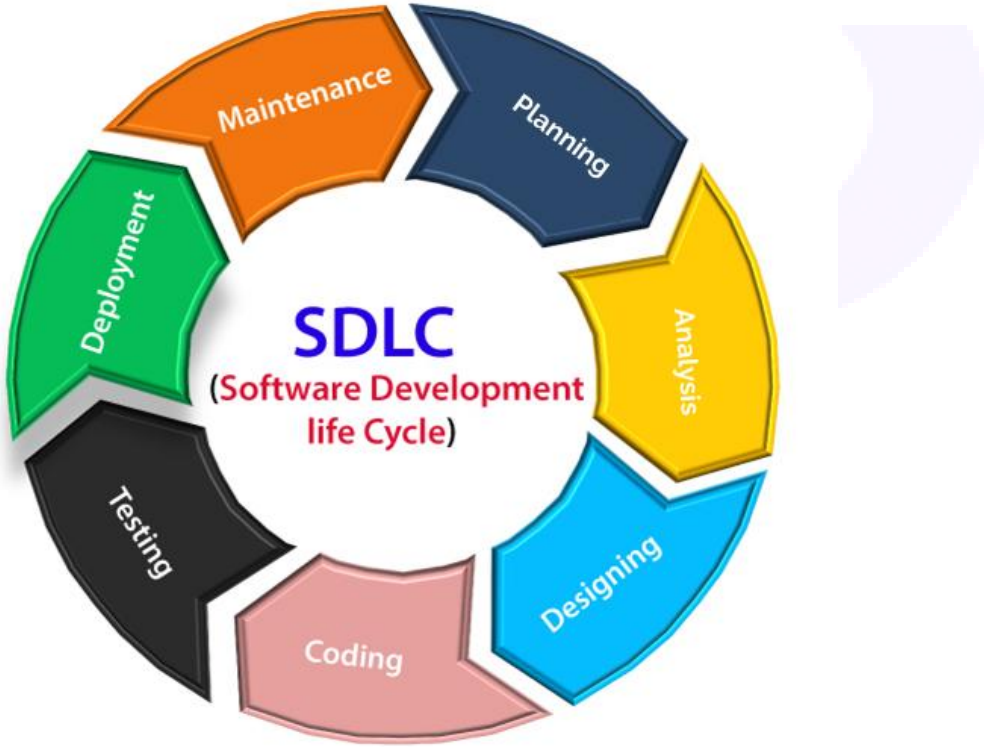
DevOps brings everyone together – from business to developers to operations – to deliver high quality software faster and more reliably.

Collaboration
Automation
Continuous Improvement

What is SDLC?

The SDLC is an iterative process that allows for feedback and adjustments to be made at each phase, to ensure that the final product meets the needs of the stakeholders and is of high quality.

Software Development Life Cycle (SDLC) is a systematic way of developing quality software.
It provides an organized plan for breaking down the task of program development into manageable chunks, each of which must be completed before moving on to the next phase.



Common SDLC Models:

1. Waterfall Model
2. V-Model (Verification and Validation)
3. Iterative & Incremental Model
4. Spiral Model
5. Agile Model
6. DevOps Model

1. Waterfall Model

Waterfall Model

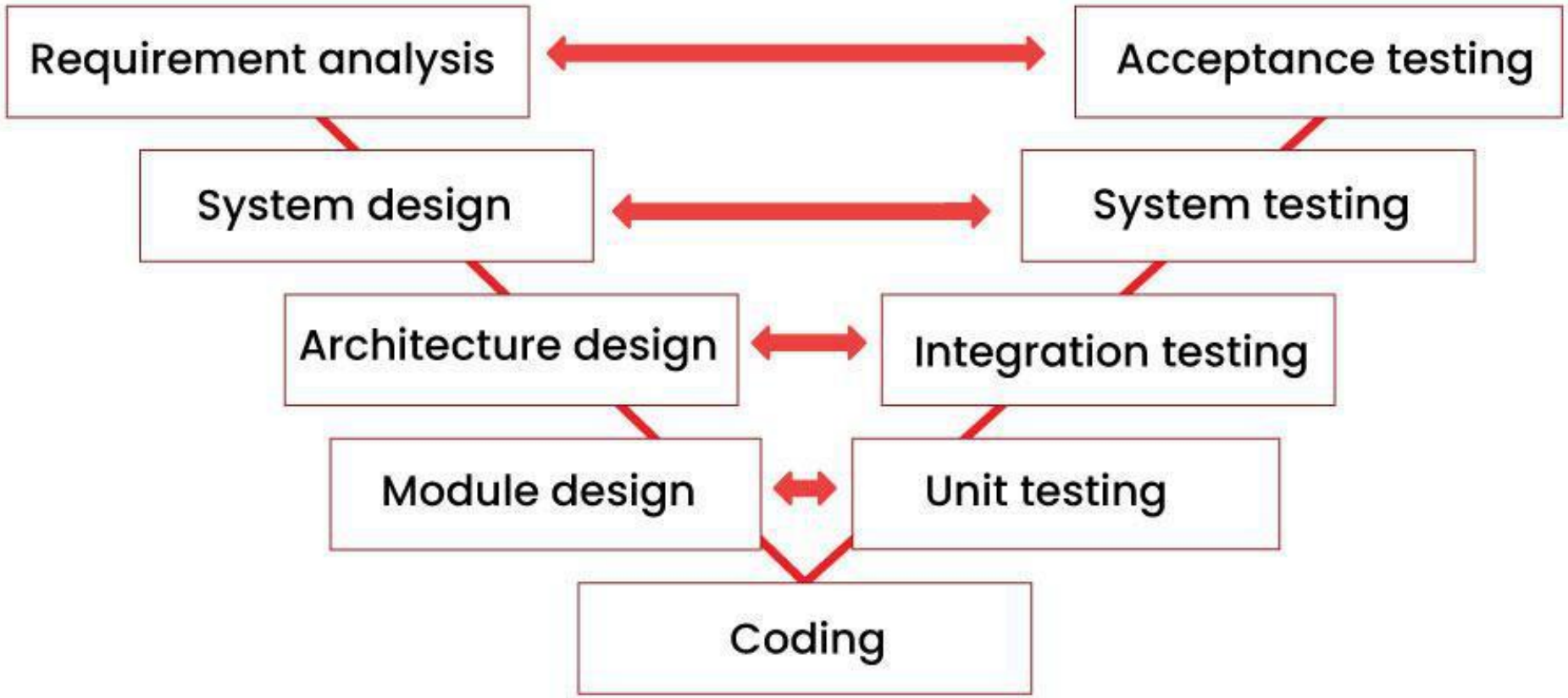
Waterfall model is introduced by Royce in year 1970. Waterfall model follows the SDLC approach and states that "the phases are organized in a linear order and the output of one phase becomes the input for the next phase."

The waterfall model is a sequential (non-iterative) design process, used in software development processes, in which progress is seen as flowing steadily downwards (like a waterfall) through the phases of conception, initiation, analysis, design, construction, testing, production/implementation and maintenance.

1. Waterfall Model

Advantages	Disadvantages
• Works well when the requirements are clear from the start of the project. • Project risk is anticipated from the beginning so gives time for mitigation. • Best if critical resource has limited availability. • Roles and responsibilities are defined early. • Reducing the risk of misunderstandings or gaps in delivery	• Time and money has to be committed early to support the planning phase. • Agreed changes in scope can be slow to implement and affect the project timeline. • Accidental scope creep adds effort and cost, decreasing the project value, so controls must be effective. • Project value can only be realized at the end of the life cycle

2. V-Model (Verification and Validation)

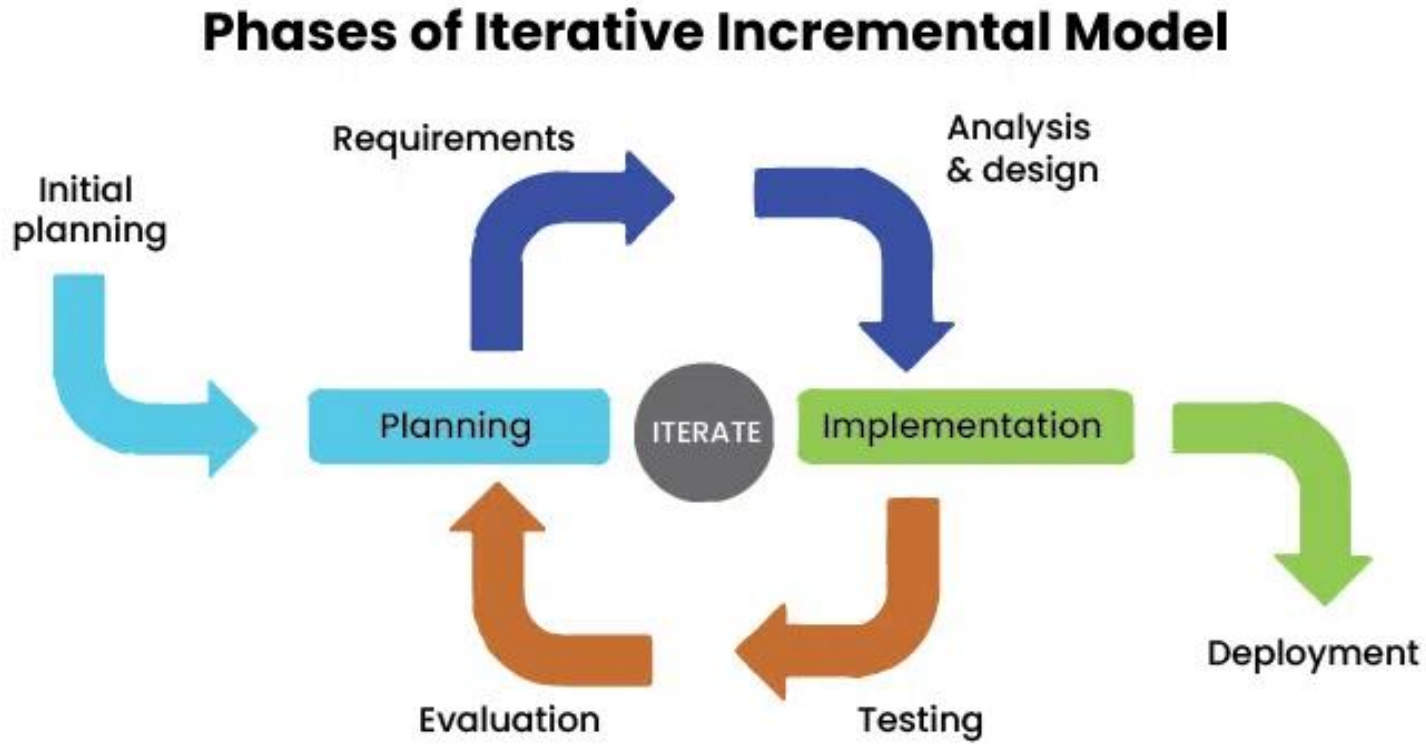


The V-Model (Verification and Validation model) is a structured software development process that pairs every design and development stage with a corresponding testing phase.
Named for its V-shaped graphical flow, it forces sequential progression where a phase begins only after the previous one completes.

2. V-Model (Verification and Validation)

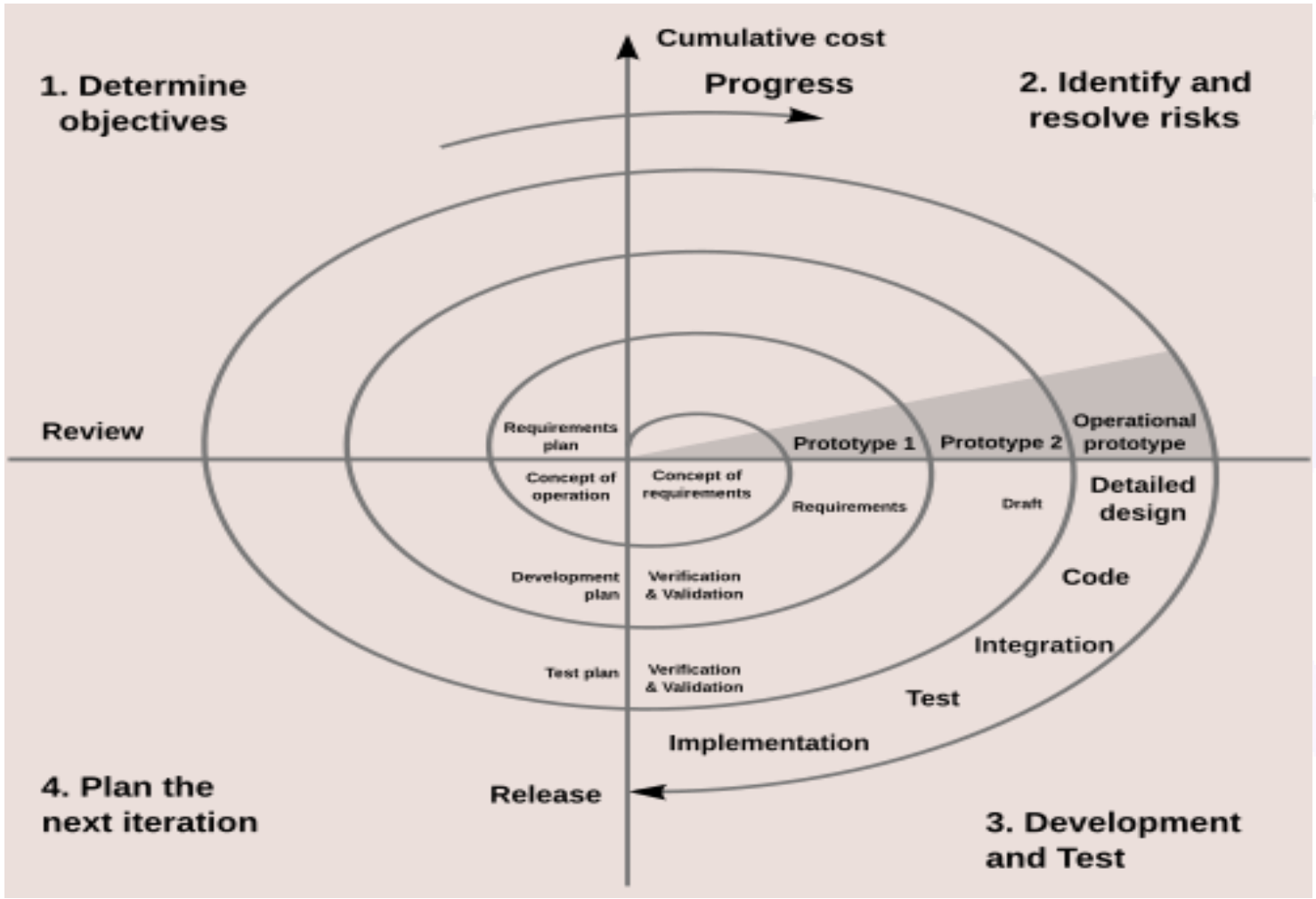
Advantages	Disadvantages
The use of the V Model is straightforward and easy for the development of software.	The V model is very rigid and hard to execute compared to other software.
The V Model architecture helps to save a lot of time compared to the general process of implementation.	The design has limited flexibility in terms of its execution. It is overall not suitable to use for building object-oriented software.
The V Model provides a proactive error tracking feature for developers.	The V Model software is developed during the phase of implementation, so no initial prototypes of the software are produced.
In the environment of the V Model, there is no problem with the downward data flow.	Both test documents and requirement documents require to be updated if there is any fault in the system.

3.Iterative & Incremental Model



Iterative development involves repeating the development cycle multiple times, with each iteration adding new features or refining existing ones. Each iteration builds upon the previous one, incorporating feedback and changes to improve the software. This approach allows for flexibility and adaptability, as requirements can evolve over time.

4. Spiral Model

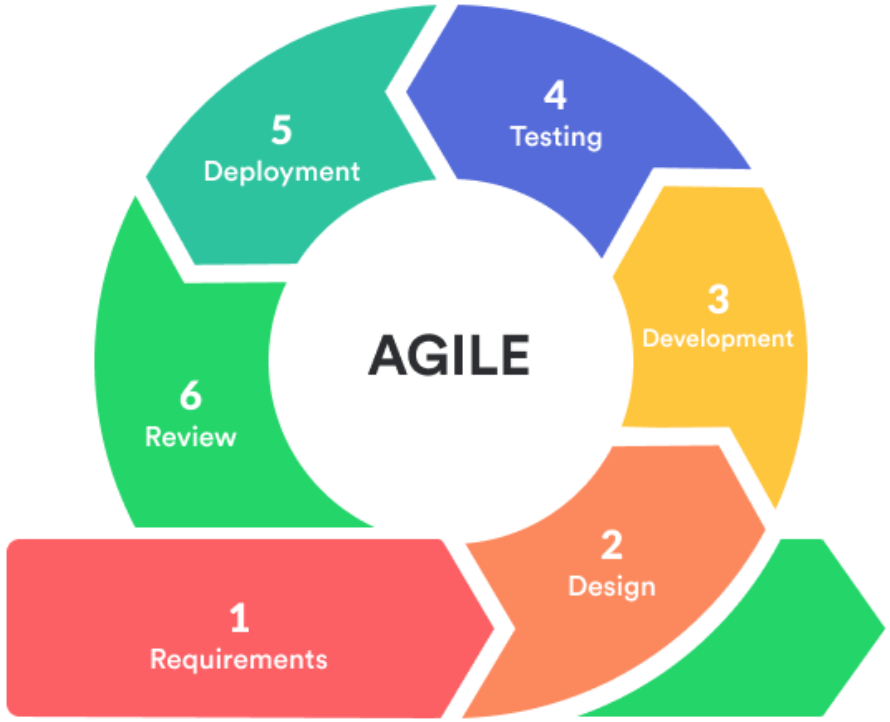


The spiral model is a risk-driven software development process model. Based on the unique risk patterns of a given project, the spiral model guides a team to adopt elements of one or more process models, such as incremental, waterfall, or evolutionary prototyping.

4. Spiral Model

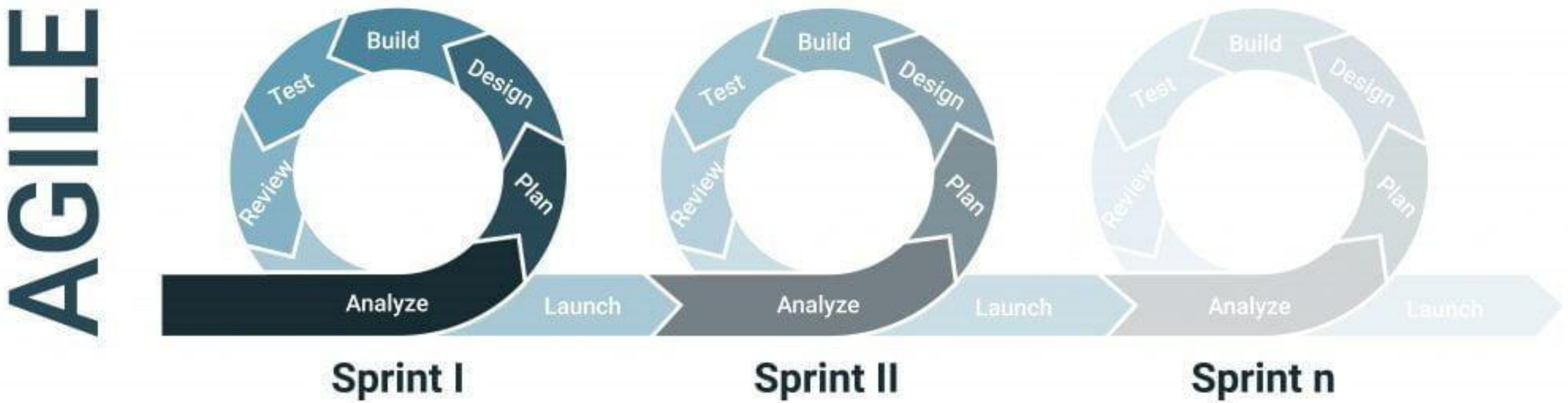
Advantages	Disadvantages
• Superior Risk Management – Risk analysis at every phase. • High Flexibility – Requirements can change later. • Customer Involvement – Feedback at each iteration. • Realistic Estimations – Improves over time	• High Cost – Expensive due to risk analysis and prototyping • Complex Management – Requires skilled professionals. • Vague Timelines – Iterations not fixed. • Excessive Documentation – Heavy documentation required

5. Agile Model



Agile Methodology is a way to manage projects by breaking them into smaller parts. It focuses on working together and making constant improvements. Teams plan, work on the project, and then review in a repeating cycle.

5. Agile Model



5. Agile Model

Advantages	Disadvantages
• Promotes team and client communication • High flexibility to adapt to changes • Speeds up development time. • Enhances client satisfaction with early, frequent deliveries. • Delivers working software regularly (in weeks)	• Poor documentation may lead to confusion. • Difficult to estimate effort for large projects. • Project may fail if stakeholders are misaligned. • Requires experienced professionals and strong technical skills. • Not suitable for beginners or teams without Agile experience.

6. DevOps Model

Understanding the Two phases before DevOps

Development (Dev): Focuses on *building* the software. This involves writing code, designing features, and fixing bugs to meet user needs. It mainly focuses on developing and bug free software.

- **Feature Creation:** A developer writes code to add a new "Dark Mode" toggle or a "Pay with Apple Pay" button to a mobile banking app.
- **Bug Fixing:** A developer reviews error logs to find out why an e-commerce checkout page crashes when a user enters a specific shipping address format, then modifies the code to fix it.
- **Code Review:** Senior developers read through code written by junior team members on platforms like GitHub to check for quality, logic errors, and readability before approving it.
- **Database Schema Updates:** A developer modifies the database structure to add a new field, such as storing a user's birthday for a marketing campaign.

Operations (Ops): Focuses on *running* the software in production.
This involves server deployment, system stability, scaling, and continuous monitoring to ensure high availability and security.

- **Cloud Infrastructure Setup:** Team configures 10 new virtual servers to host a new website.
- **Traffic Scaling:** Operation engineer sets up rules so that if a website gets a massive spike in traffic , the system automatically launches extra servers to handle the load without crashing.
- **Monitoring & Alerting:** A systems engineer configures tracking dashboards on Datadog or Prometheus.
  If a server's memory usage spikes above 90%, the system automatically sends an urgent alert to the team.
- **Security Patching:** A systems administrator logs into the company's operating systems to install critical security updates and block newly discovered vulnerabilities.

Roadblocks before DevOps

1. Different Goals (Core Conflict)

Dev wants speed → release features quickly
Ops wants stability → avoid failures

2. It Works on My Machine" Issue

Code runs fine in developer's local environment
Fails in production due to:

- Different OS
- Different libraries
- Configuration mismatch

3. Communication Gap

- Dev and Ops work separately
- No shared understanding of system

Problem:

- Miscommunication
- Slow response to issues
- Lack of collaboration

4. Manual Deployments.

- Deployments done manually by Ops
- Steps:
  - Copy files
  - Run scripts
  - Restart servers

Problem:

- Human errors
- Deployment failures

5. Environment Drift

- Dev, Test, and Prod environments are different

Problem:

- Unexpected behavior in production
- Hard to reproduce issues

6. Lack of Visibility

- Ops monitors production
- Dev has no insight into live system

Problem:

Dev cannot understand real issues
Troubleshooting takes longer

Advances of DevOps Engineer

Fast Delivery
Automation of build, test, and deployment pipelines enables quicker and frequent releases of applications.

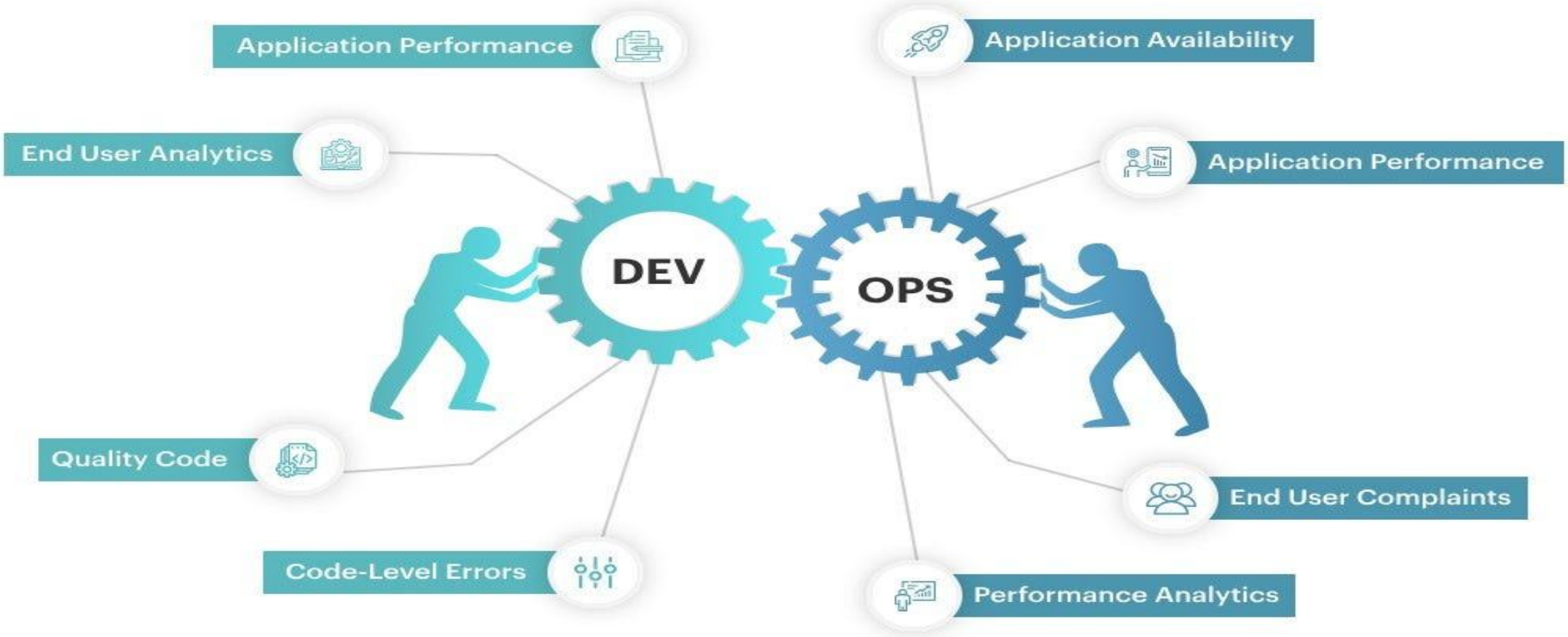
Higher Quality
Continuous integration and automated testing ensure bugs are detected early, improving overall software quality.

Reduced OPEX (Operational Expenditure)
Automation and efficient resource management reduce manual effort and ongoing operational costs.

Reduced CAPEX (Capital Expenditure)
Cloud and Infrastructure as Code minimize the need for heavy upfront investment in physical infrastructure.

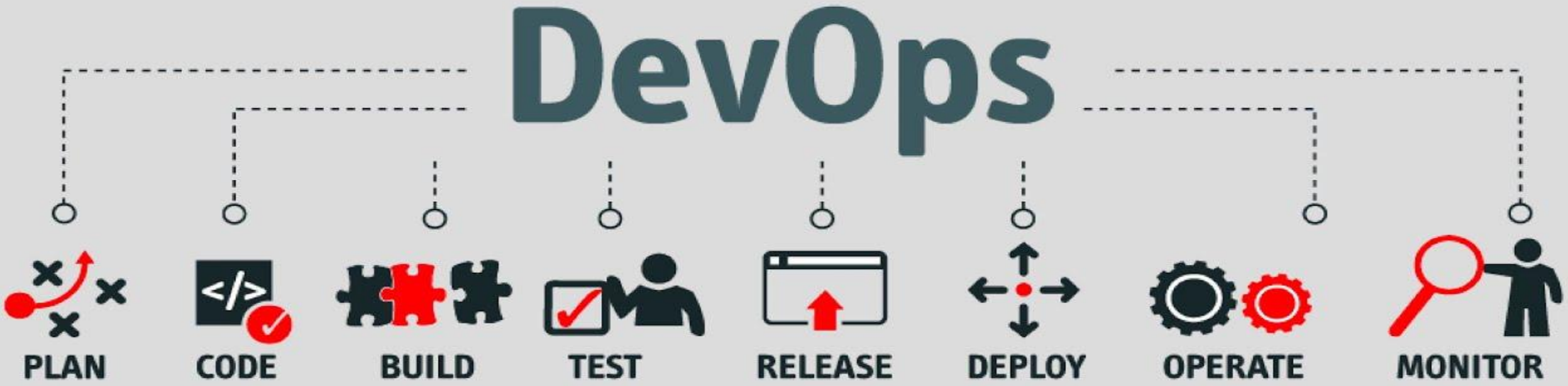
Reduced Outages
Monitoring, automated deployments, and quick rollback mechanisms help prevent and quickly resolve system failures.

6. DevOps Model



DevOps is a modern approach to software development that brings development and operations teams together to deliver applications faster and more reliably. It focuses on collaboration, automation, and continuous improvement across the software lifecycle.

6. DevOps Model



DevOps is a modern approach to software development that brings development and operations teams together to deliver applications faster and more reliably. It focuses on collaboration, automation, and continuous improvement across the software lifecycle.